

네트워크 2 - TCP

남궁명수

[TCP]

TCP(Transmission Control Protocol)는 애플리케이션 사이에 연결 지향적이고 신뢰성 있는 바이트 스트림 통신을 제공하는 전송 계층 프로토콜이다.

TCP의 핵심 기능은 다음과 같다.

- 연결 설정 및 종료
- 데이터 순서 보장
- 데이터 손실 시 재전송
- 중복 데이터 제거
- 오류 검출
- 흐름 제어
- 혼잡 제어
- 전이중 통신

TCP는 메시지 경계를 보존하지 않는 바이트 스트림 방식이다.

예를 들어, 송신자가 다음처럼 두 번 전송해도:

```
send("Hello")  
send("World")
```

수신자는 다음과 같이 받을 수 있다.

```
recv() → "HelloWorld"
```

따라서 TCP 애플리케이션은 길이 필드, 구분자 같은 방식으로 메시지 경계를 직접 정의해야 한다.

[1] [TCP 연결 과정]

전체 TCP 통신은 크게 세 단계로 나뉜다.

1. 연결 설정
3-Way Handshake
2. 데이터 송수신
Sequence Number
ACK
재전송
흐름 제어
혼잡 제어
3. 연결 종료
일반적으로 4개의 세그먼트 교환

서버는 먼저 소켓을 생성하고 연결 요청을 기다린다.

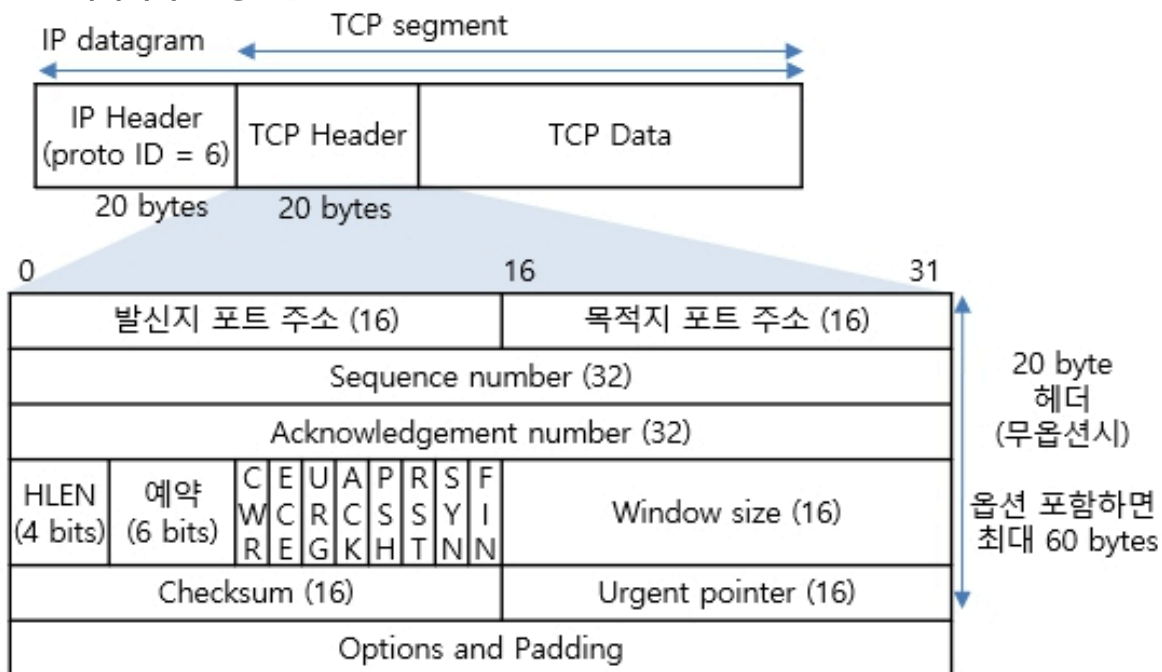
```
socket();  
bind();  
listen();  
accept();
```

클라이언트는 소켓을 생성하고, 서버에 연결을 요청한다.

```
socket();  
connect();
```

이때 클라이언트의 connect()와 서버의 accept() 사이에 3-Way Handshake가 진행된다.
연결이 완료되면 양쪽은 send()/recv() 또는 write()/read()를 통해 데이터를 송수신한다.

[2] [TCP 헤더의 주요 정보]



※ [참고] TCP 헤더 크기 : (기본 20 ~ 최대 60 바이트)

참고 자료: http://www.ktword.co.kr/test/view/view.php?m_temp1=1889

TCP 연결 과정을 이해하려면 다음 필드가 중요하다.

필드	의미
Source Port	송신 프로세스의 포트
Destination Port	수신 프로세스의 포트
Sequence Number	현재 전송하는 데이터의 바이트 번호
Acknowledgment Number	다음에 받고 싶은 바이트 번호
Flags	SYN, ACK, FIN, RST 등의 제어 정보
Window Size	현재 수신자가 받을 수 있는 공간
Checksum	TCP 헤더와 데이터의 오류 검사

주요 TCP 플래그는 다음과 같다.

플래그	의미
SYN	연결 설정 및 초기 시퀀스 번호 동기화
ACK	확인 응답 번호가 유효함
FIN	더 이상 보낼 데이터가 없음을 알림
RST	연결을 즉시 강제로 초기화
PSH	데이터를 애플리케이션에 빠르게 전달하도록 요청
URG	긴급 데이터가 포함되어 있음을 표시

[ACK와 Ack의 차이]

TCP 헤더 안에서 여러 정보가 동시에 들어 있다.

TCP 헤더

- └ Flags: SYN, ACK, FIN 등
- └ Sequence Number
- └ Acknowledgment Number

ACK = 1: ACK 플래그

ACK는 TCP 헤더에 있는 스위치 또는 체크 표시이다.

ACK = 1

- TCP 세그먼트의 Acknowledgment Number 필드가 유효하다.

ACK = 0

- Acknowledgment Number 값은 확인 응답의 의미로 사용되지 않는다.

Ack = 101 // Acknowledgment Number

Ack = 101은 구체적인 응답 번호이다

의미는 다음과 같다.

- 100번까지 받았고, 다음에는 101번부터 보내주세요.

따라서 다음 두 정보가 함께 사용된다.

ACK = 1 // 확인 응답 번호가 유효하다는 표시

Ack = 101 // 다음에는 101번부터 보내 달라는 실제 값

쉽게 비유하면 다음과 같다.

ACK = 1 → “확인 응답입니다”라는 체크박스

Ack = 101 → “101번부터 보내주세요”라는 구체적인 내용

[Seq는 무엇인가?]

Seq는 Sequence Number, 즉 내가 보내는 데이터의 번호이다.

TCP는 데이터를 단순히 덩어리로만 관리하는 게 아니라 바이트마다 순서를 붙여서 관리한다.

예를 들어, 100번부터 시작하는 5바이트를 보낸다면:

Seq = 100

Data = "HELLO"

각 바이트는 개념적으로 다음 번호를 가진다.

H	E	L	L	O
100	101	102	103	104

수신자가 전부 정상적으로 받으면 다음과 같이 응답한다.

ACK = 1

Ack = 105

- 104번까지 정상적으로 받았다. 다음에는 105번부터 보내주세요.

따라서 Ack는 보통 마지막으로 받은 번호 + 1이다.

받은 마지막 번호: 104

다음에 받을 번호: 105

Ack = 105

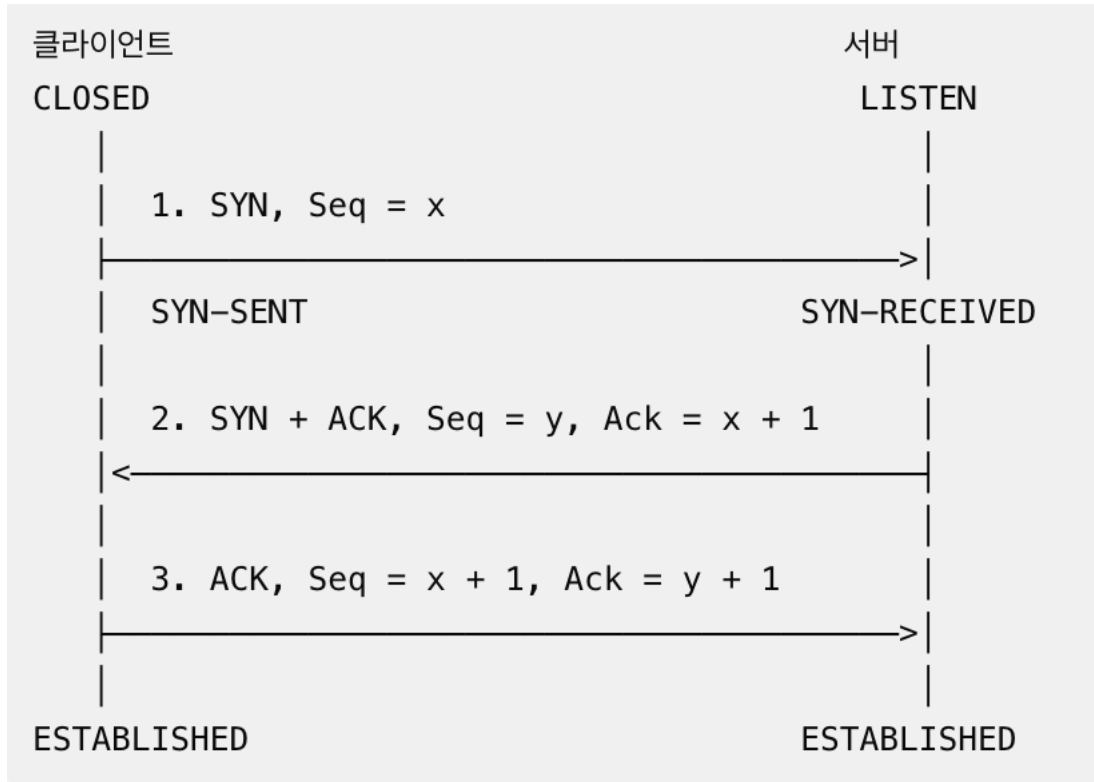
[3] [3-Way Handshake]

3-Way Handshake는 클라이언트와 서버가 TCP 통신을 시작하기 전에 연결을 설정하는 과정이다.

이 과정을 통해 다음 내용을 확인한다.

- 클라이언트와 서버가 서로 통신 가능한 상태인지 확인
- 양방향 통신이 가능한지 확인
- 서로의 초기 시퀀스 번호(ISN) 교환
- TCP 옵션 협상

[연결 과정]



[용어 정리]

먼저 SYN, ACK, Seq, Ack를 구분해야 한다.

- SYN = 연결을 요청하고 초기 시퀀스 번호를 전달하는 플래그
- ACK = Acknowledgment Number 필드가 유효하다는 플래그
- Seq = 자신이 보내는 데이터의 시작 번호
- Ack = 상대방이 다음에 보내야 할 번호

[1단계: 클라이언트 -> 서버]

SYN = 1
Seq = x

- SYN = 1 : TCP 연결을 요청한다.
- Seq = x : 클라이언트가 사용할 초기 시퀀스 번호는 x이다.
- x는 클라이언트가 임의로 선택한 ISN(Initial Sequence Number)

클라이언트 상태:

CLOSED → SYN-SENT

[2단계: 서버 -> 클라이언트]

SYN = 1
ACK = 1
Seq = y
Ack = x + 1

- SYN = 1 : 서버도 클라이언트와 연결하겠다는 의미다.
- ACK = 1 : Ack 필드에 유효한 확인 응답 번호가 들어 있다는 의미다.
- Seq = y : 서버가 선택한 초기 시퀀스 번호는 y다.
- Ack = x + 1 : 클라이언트가 보낸 Seq = x의 SYN을 받았으므로, 다음에는 x+1번부터 보내라는 의미다.

여기서 Ack가 x가 아니라 x+1인 이유는 SYN이 시퀀스 번호 하나를 사용하기 때문이다.

서버의 응답을 말로 표현하면 다음과 같다.

- 클라이언트야, 너가 보낸 x번 SYN을 받았어. 다음에는 x+1번부터 보내줘. 그리고 내 시작 번호는 y야.

서버 상태

LISTEN → SYN-RECEIVED

[3단계: 클라이언트 -> 서버]

ACK = 1
Seq = x + 1
Ack = y + 1

- ACK = 1 : Ack 필드에 유효한 확인 응답 번호가 들어 있다는 의미다.
- Seq = x + 1 : 클라이언트의 SYN이 x번을 사용했으므로, 현재 클라이언트의 다음 시퀀스 번호는 x + 1이다.
- Ack = y + 1 : 서버가 Seq = y로 보낸 SYN을 받았으므로, 서버는 다음에 y+1번부터 보내야 한다.

클라이언트의 응답을 말로 표현하면 다음과 같다.

- 서버야, 너가 보낸 y번 SYN을 받았어. 다음에는 y + 1부터 보내줘

이 ACK가 서버에 도착하면 양쪽 모두 연결이 설정된다.

클라이언트: ESTABLISHED

서버: ESTABLISHED

[SYN과 FIN이 시퀀스 번호를 1개 소비하는 이유]

SYN과 FIN은 일반 데이터가 아니지만 TCP 연결 상태를 변경하는 중요한 제어 정보이므로 시퀀스 번호 공간에서 1바이트를 차지하는 것처럼 관리된다.

따라서

SYN Seq = 100

다음 Seq = 101

반면 데이터가 없는 순수 ACK는 일반적으로 시퀀스 번호를 소비하지 않는다.

[왜 2번이 아니라 3번이 필요한가?]

TCP는 전이중 통신이므로 양쪽 방향의 통신 가능 여부를 모두 확인해야 한다.

1단계: 클라이언트 → 서버 통신 가능

2단계: 서버가 클라이언트 요청을 받았으며 서버 → 클라이언트도 통신 가능

3단계: 클라이언트가 서버 응답까지 정상적으로 받았음을 서버가 확인

만약 마지막 ACK이 없다면 서버는 클라이언트가 자신의 SYN+ACK를 받았는지 확인할 수 없다.

또한 과거 네트워크에 지연되어 있던 오래된 SYN이 뒤늦게 도착했을 때, 서버가 잘못된 연결을 확정하는 문제도 줄일 수 있다.

[3-Way Handshake에서 협상하는 정보]

양쪽은 SYN 세그먼트의 TCP 옵션을 통해 다음 정보를 협상할 수 있다.

MSS: 한 TCP 세그먼트에 담을 최대 데이터 크기

Window Scale: 수신 윈도우 확장

SACK Permitted: 선택적 확인 응답 지원 여부

TimeStamp: 시간 측정과 오래된 세그먼트 구분에 사용

[4] [TCP 데이터 전송과 신뢰성]

TCP는 데이터를 바이트 단위의 시퀀스 번호로 관리한다.

예를 들어, 송신자가 시퀀스 번호 1001부터 500바이트를 보냈다고 하자.

```
Seq = 1001  
Data Length = 500
```

수신자가 정상적으로 받으면 다음과 같이 응답한다.

```
Ack = 1501
```

- 1500번 바이트까지 정상적으로 받았으니 다음에는 1501번부터 보내달라.

TCP의 ACK는 기본적으로 누적 확인 응답이다.

```
ACK =1501
```

- 1500번까지의 모든 바이트를 연속적으로 받았다는 의미다.

[손실된 데이터 재전송]

TCP는 대표적으로 두 방식으로 손실을 판단한다.

(1) 재전송 타이머 만료

송신자는 데이터를 보낸 후 ACK를 기다린다.

일정 시간 동안 ACK가 오지 않으면 데이터가 손실됐다고 판단하여 재전송한다.

이 시간을 RTO(Retransmission Timeout)라고 한다.

(2) 중복 ACK 수신

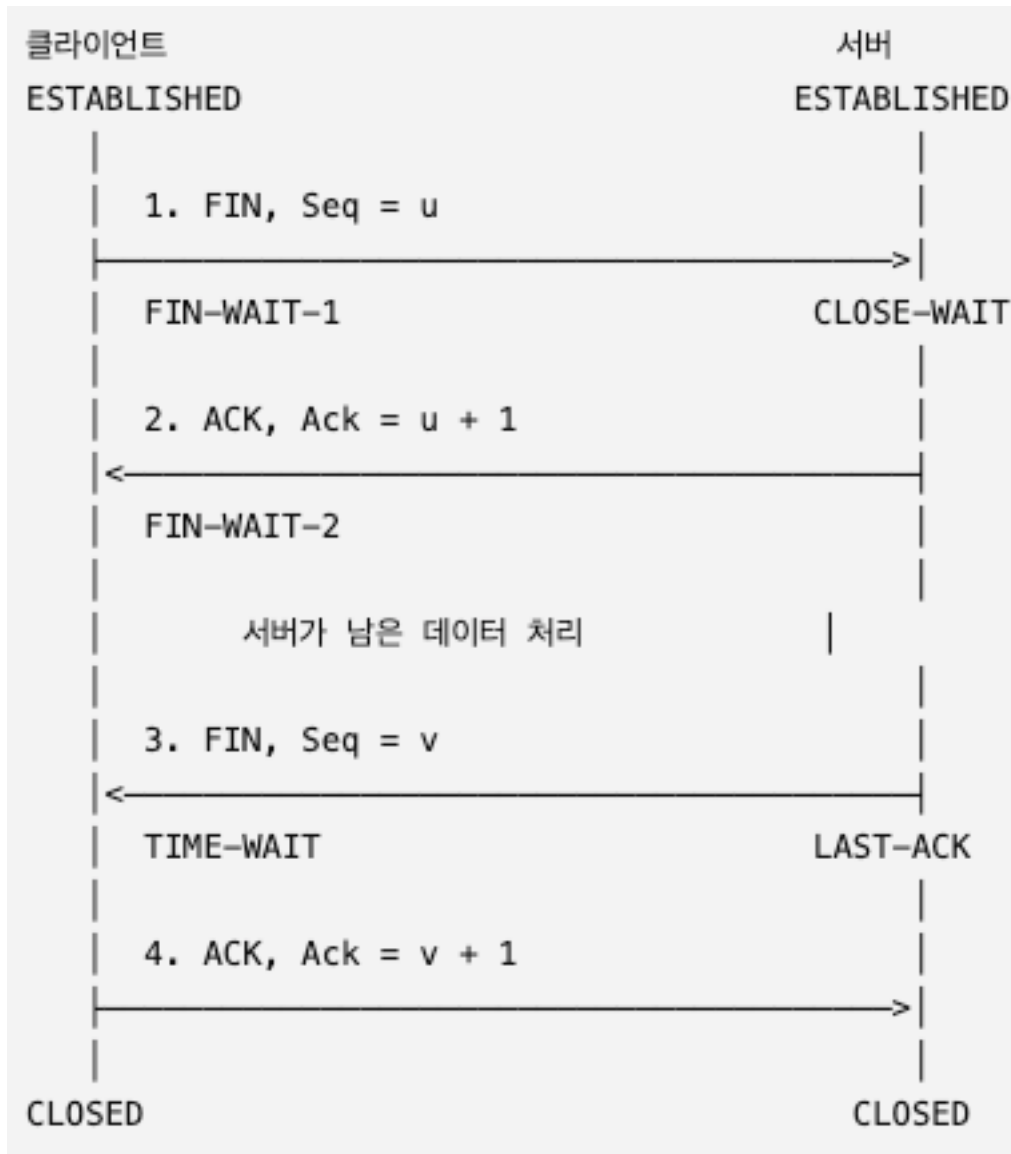
중간 세그먼트 하나가 유실되면 수신자는 계속 같은 ACK 번호를 보낼 수 있다.

송신자가 일반적으로 같은 ACK를 여러 번 연속으로 받으면 타이머 만료를 기다리지 않고 손실된 세그먼트를 재전송한다. 이를 Fast Retransmit이라고 한다.

[5] [TCP 연결 종료]

TCP는 전이중 통신이기 때문에 클라이언트와 서버가 데이터를 보내는 방향을 각각 독립적으로 종료한다. 따라서 정상 종료 과정에서는 일반적으로 FIN과 ACK가 네 단계로 교환된다.

흔히 4-Way Handshake라고 부르지만, 정확히는 TCP 연결 종료를 위한 FIN/ACK 교환 과정이라고 보면 된다.



[1단계: 먼저 종료하는 쪽 -> 상대방]

FIN = 1
Seq = u

먼저 종료하는 쪽이 더 이상 보낼 데이터가 없다고 알린다.

클라이언트가 먼저 종료했다고 가정하면:

ESTABLISHED → FIN-WAIT-1

[2단계: 상대방 -> 먼저 종료한 쪽]

ACK = 1
Ack = u + 1

서버가 FIN을 받았다고 응답한다.

상태 변화

클라이언트: FIN-WAIT-1 → FIN-WAIT-2
서버: ESTABLISHED → CLOSE-WAIT

이 시점에 클라이언트는 더 이상 데이터를 보내지 않지만, 서버가 보내는 데이터는 받을 수 있다.
이를 Half-Close 상태라고 한다.

[3단계: 상대방 -> 먼저 종료한 쪽]

서버가 남은 데이터를 모두 처리한 다음 자신의 통신 방향도 종료한다.

FIN = 1
Seq = v

서버 상태:

CLOSE-WAIT → LAST-ACK

[4단계: 먼저 종료한 쪽 -> 상대방]

ACK = 1
Ack = v + 1

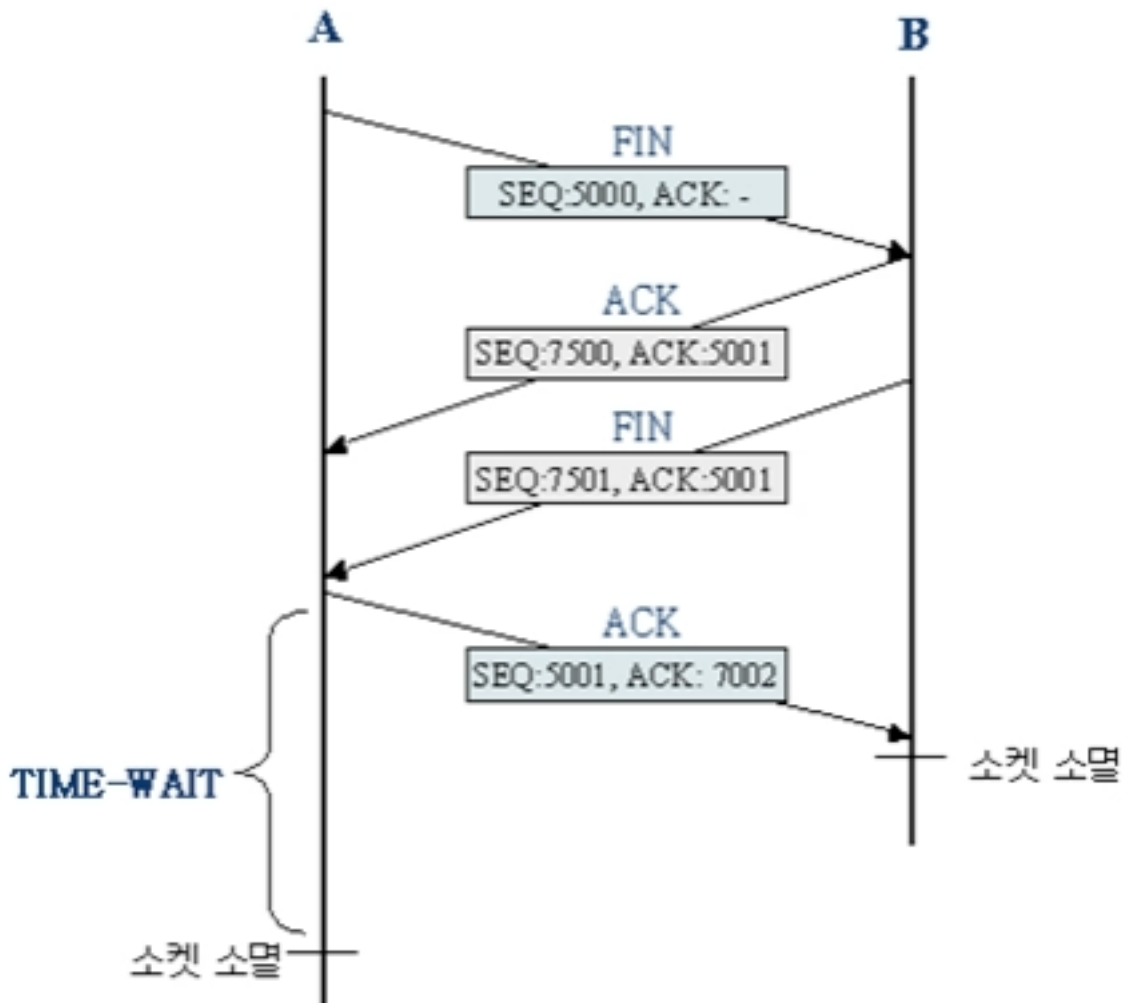
클라이언트가 서버의 FIN을 확인했다고 ACK를 보낸다.

서버가 ACK를 받으면 연결을 완전히 종료한다.

서버: LAST-ACK → CLOSED

클라이언트는 즉시 종료되지 않고 TIME-WAIT 상태에 머문다.

[6] [TIME_WAIT]



일반적으로 마지막 ACK를 보낸 쪽, 즉 능동적으로 연결 종료를 시작한 쪽이 TIME_WAIT 상태에 들어간다.

TIME_WAIT의 유지 시간은 일반적으로 $2 \times \text{MSL}$ 로 설명한다.

MSL: 세그먼트가 네트워크에서 존재할 수 있는 최대 시간
 실제 운영체제의 구체적인 유지 시간은 구현과 설정에 따라 다름

[TIME_WAIT가 필요한 이유는 두 가지이다]

[1. 마지막 ACK 재전송]

마지막 ACK가 유실되면 서버는 자신의 FIN이 전달되지 않았다고 판단하여 FIN을 다시 보낸다. 클라이언트가 일정 시간 동안 상태를 유지해야 다시 도착한 FIN에 ACK를 재전송할 수 있다.

[2. 이전 연결의 지연 패킷 제거]

동일한 IP와 포트 조합으로 새 연결이 만들어졌을 때, 이전 연결에서 늦게 도착한 패킷이 새로운 연결의 데이터로 오인되는 것을 방지한다.

TCP 연결은 다음 네 가지 값으로 구분한다.

출발지 IP
출발지 Port
목적지 IP
목적지 Port

이를 **4-Tuple**이라고 한다.

[7] [CLOSE_WAIT가 많이 쌓이는 이유]

CLOSE_WAIT는 상대방의 FIN을 받은 상태다.

상대방: 연결을 종료하겠다고 FIN 전송
내 커널: FIN을 받고 ACK 전송
내 프로세스: 아직 소켓을 close()하지 않음

따라서 CLOSE_WAIT가 계속 쌓인다면 보통 애플리케이션이 연결 종료를 제대로 처리하지 않은 것이다.

예를 들면:

- 예외 발생 후 close()를 호출하지 않음
- 연결 관리 코드 누락
- 고루틴이나 스레드가 블로킹된 상태
- 응답을 기다리며 종료하지 못함

반면 TIME_WAIT가 많은 것은 반드시 버그라는 의미는 아니다. 짧은 연결을 많이 생성하고 정상 종료하면 자연스럽게 많이 발생할 수 있다.

[8] [FIN과 RST의 차이]

FIN - 정상적인 연결 종료에 사용한다.

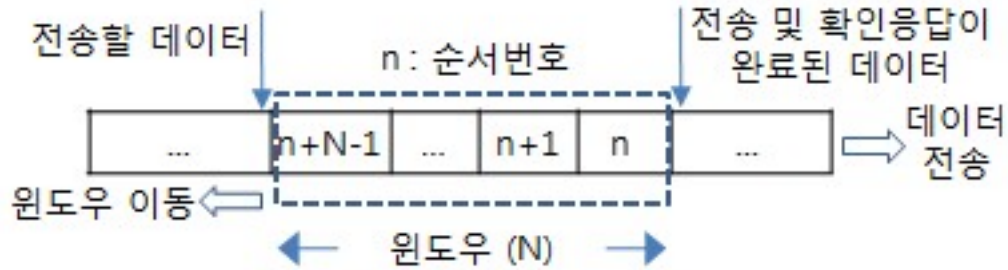
- 남은 데이터를 처리할 수 있음
- 상대방에게 종료 사실을 알림
- ACK를 주고받으며 단계적으로 종료

RST - 연결을 즉시 강제로 종료한다.

- 해당 포트를 수신하는 프로그램이 없을 때
- 유효하지 않은 연결에 데이터가 도착했을 때
- 애플리케이션이 연결을 비정상적으로 종료할 때
- 연결 상태가 심각하게 잘못됐을 때

FIN = 정상적인 종료
RST = 즉시 연결 초기화

[9] [Flow Control - 흐름 제어]



흐름 제어는 송신자가 수신자의 처리 능력을 초과하여 데이터를 보내지 않도록 조절하는 기능이다.
핵심 질문은 다음과 같다.

수신자가 지금 데이터를 얼마나 더 받을 수 있는가?

[흐름 제어가 필요한 이유]

송신자는 매우 빠르고 수신자는 느릴 수 있다.

송신자 전송 속도 > 수신자의 데이터 처리 속도

수신 버퍼가 가득 차면 새로 도착한 데이터를 저장할 공간이 없어 데이터가 손실될 수 있다.
TCP는 이를 방지하기 위해 수신자가 자신의 남은 버퍼 크기를 송신자에게 알린다.

이를 수신 원도우 또는 rwnd(Receiver Window)라고 한다.

수신 버퍼 전체 크기: 10,000바이트

현재 사용 중인 크기: 6,000바이트

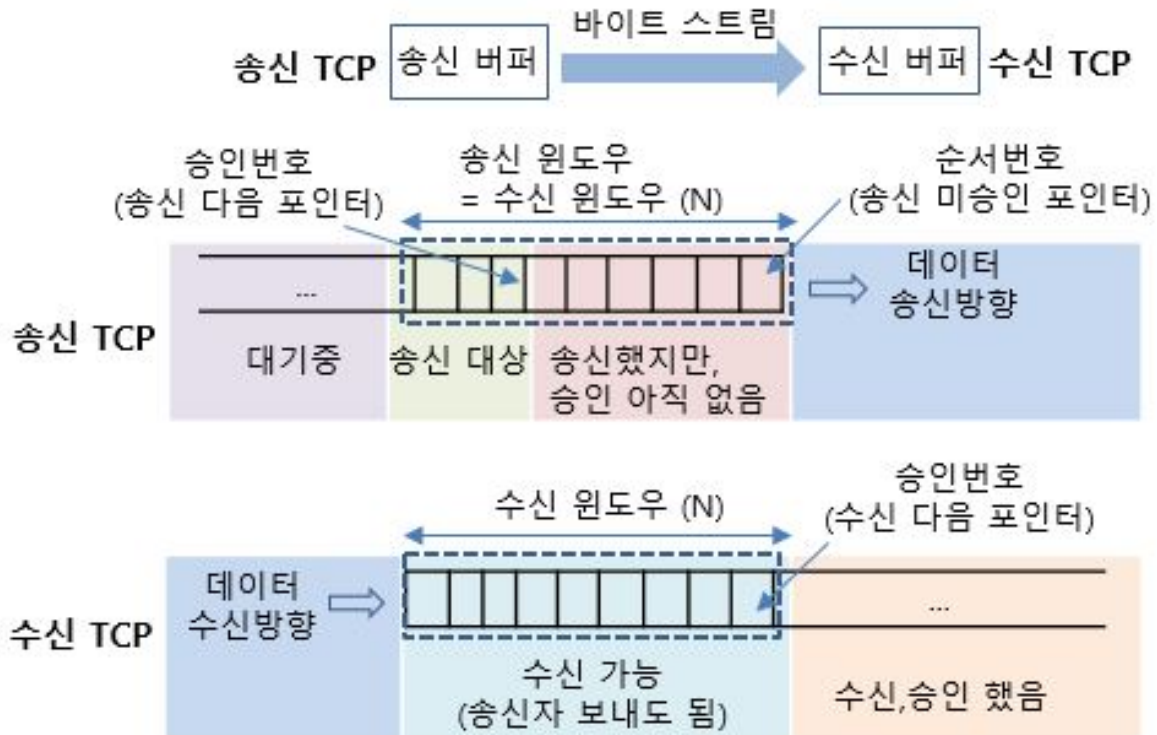
남은 공간(rwnd): 4,000바이트

수신자는 TCP 헤더의 Window Size 필드를 이용해 다음과 같이 알린다.

현재 4,000바이트를 더 받을 수 있습니다.

송신자는 확인 응답을 받지 않은 데이터의 양이 rwnd를 초과하지 않도록 조절한다.

[Sliding Window]



TCP는 매 세그먼트마다 하나를 보내고 ACK를 기다리지 않는다.
수신자가 허용한 윈도우 범위 안에서 여러 세그먼트를 연속으로 전송할 수 있다.

[아직 전송 불가] [전송 가능] [전송했지만 ACK 대기] [ACK 완료]

ACK를 받으면 전송 가능한 범위가 왼쪽으로 이동한다. 이를 **슬라이딩 윈도우**라고 한다.

즉,
rwnd는 수신자가 허용한 데이터 전송량이고, 슬라이딩 윈도우는 그 허용 범위 안에서 여러 데이터를 보내고 ACK에 따라 범위를 이동시키는 전송 방식이다.

[Stop-and-Wait와 비교]

Stop-and-Wait:

데이터 전송 → ACK 대기 → 데이터 전송 → ACK 대기

Sliding Window:

데이터 전송 → 데이터 전송 → 데이터 전송 → ACK 수신

슬라이딩 윈도우는 네트워크를 기다리는 동안 여러 데이터를 전송할 수 있으므로 전송 효율을 높인다.

[zero Window]

수신 버퍼가 가득 차면 수신자는 다음과 같이 알릴 수 있다.

rwnd = 0

송신자는 일반 데이터 전송을 멈춘다. 이후 작은 Window Probe를 주기적으로 보내 수신 버퍼에 공간이 생겼는지 확인한다.

[10] [Congestion Control - 혼잡 제어]

혼잡 제어는 송신자가 네트워크가 처리할 수 있는 양보다 많은 데이터를 보내지 않도록 조절하는 기능이다.

핵심 질문은 다음과 같다.

현재 네트워크가 데이터를 얼마나 감당할 수 있는가?

[네트워크 혼잡이란?]

여러 송신자가 라우터나 네트워크 링크에 너무 많은 패킷을 보내면 라우터의 큐가 가득 찬다.

여러 송신자
↓
라우터 입력 패킷 증가
↓
라우터 큐 포화
↓
지연 증가 및 패킷 손실
↓
재전송 증가
↓
혼잡 악화

TCP는 네트워크의 실제 용량을 정확히 알 수 없으므로 ACK, 패킷 손실, 왕복 시간 등을 보고 혼잡 상태를 추정한다. 혼잡 제어에서 송신자가 관리하는 전송 한도를 cwnd(Congestion Window)라고 한다.

[네트워크 링크]

데이터를 전송하기 위해 두 개 이상의 장치(노드)를 연결하는 물리적인 통신 경로나 유무선 통신 채널을 의미한다. 노드가 컴퓨터나 라우터 같은 기기라면, 링크는 이들을 잇는 랜선이나 무선 신호에 해당한다.

[Link Up/Down]

네트워크 장비의 상태를 나타내는 지표로, Link Up은 물리적 연결이 정상 작동하여 통신이 가능한 상태, Link Down은 연결이 끊어진 상태를 말한다.

[11] [rwnd와 cwnd의 관계]

TCP 송신자가 실제로 보낼 수 있는 데이터 양은 수신 윈도우와 혼잡 윈도우 중 더 작은 값으로 제한된다.

실제 전송 윈도우 = $\min(rwnd, cwnd)$

예를 들어:

rwnd = 10,000바이트
cwnd = 4,000바이트

수신자는 10,000바이트를 받을 수 있지만 네트워크 혼잡 상태를 고려하면 4,000바이트만 전송한다.

반대라면:

rwnd = 2,000바이트
cwnd = 8,000바이트

네트워크에는 여유가 있어도 수신자가 2,000바이트밖에 받을 수 없으므로 2,000바이트만 전송한다.

구분	Flow Control	Congestion Control
보호 대상	수신자	네트워크
핵심 변수	rwnd	cwnd
목적	수신 버퍼 초과 방지	네트워크 혼잡 방지
판단 정보	수신자가 알린 버퍼 공간	ACK, 손실, RTT 등
핵심 질문	수신자가 얼마나 받을 수 있는가?	네트워크가 얼마나 처리할 수 있는가?

[12] [대표적인 혼잡 제어 과정]

전통적인 TCP 혼잡 제어는 다음 원리를 사용한다.

- Slow Start(느린 시작)
- Congestion Avoidance(혼잡 회피)
- Fast Retransmit(빠른 재전송)
- Fast Recovery(빠른 회복)

구체적인 동작은 TCP 혼잡 제어 알고리즘에 따라 달라질 수 있지만, 기본 원리는 알아둬야 한다.

[Slow Start (느린시작)]

연결 직후부터 많은 데이터를 보내면 네트워크를 혼잡하게 만들 수 있다.

따라서 작은 혼잡 윈도우에서 시작해 ACK가 정상적으로 도착하면 전송량을 빠르게 증가시킨다.

예를 들어 단순화하면:

1 MSS → 2 MSS → 4 MSS → 8 MSS → 16 MSS

여기서 MSS는 TCP 세그먼트 하나에 넣을 수 있는 최대 데이터 크기다.

Slow Start라는 이름과 달리 전송량은 대략 지수적으로 빠르게 증가한다.

여기서 “Slow”는 네트워크 용량을 알지 못한 상태에서 한꺼번에 최대치로 보내지 않는다는 의미에 가깝다.

[Congestion Avoidance (혼잡 회피)]

혼잡 윈도우가 임계값인 ssthresh에 도달하면 증가 속도를 완만하게 바꾼다.

단순화하면:

16 MSS → 17 MSS → 18 MSS → 19 MSS

전송량을 계속 늘리기는 하지만 혼잡을 피하기 위해 대략 선형적으로 증가시킨다.

ssthresh(Slow Start Threshold)

네트워크에서 TCP 혼잡 제어 알고리즘이 동작할 때 ‘느린 시작’ 단계를 멈추고 ‘혼잡 회피’ 단계로 전환하는 기준점

[Fast Retransmit (빠른 재전송)]

송신자가 중간 세그먼트를 받지 못했지만 그 이후 세그먼트를 받으면 같은 ACK 번호를 반복해서 보낸다.
송신자가 일반적으로 3개의 중복 ACK를 받으면 해당 세그먼트가 손실됐다고 판단하고 RTO 만료를 기다리지 않고 재전송한다.

```
Seq 1000 수신
Seq 2000 손실
Seq 3000 수신 → ACK 2000
Seq 4000 수신 → ACK 2000
Seq 5000 수신 → ACK 2000

중복 ACK 누적 → Seq 2000 재전송
```

[Fast Recovery (빠른 회복)]

중복 ACK를 통해 손실을 발견했다는 것은 일부 패킷은 계속 목적지에 도착하고 있다는 의미다.
따라서 전송량을 무조건 처음 수준까지 낮추기보다는 혼잡 윈도우를 감소시킨 후 비교적 빠르게 전송을 회복한다.

반면 재전송 타이머까지 만료됐다면 네트워크 혼잡이 더 심한 것으로 보고 일반적으로 전송량을 더 크게 줄인다.

[13] [TCP 연결 과정과 시스템 콜]

클라이언트에서 다음 코드가 실행된다고 하자.

```
conn, err := net.Dial("tcp", "google.com:80")
```

내부적으로는 개념상 다음 과정이 일어난다.

1. DNS 조회
2. socket()으로 TCP 소켓 생성
3. connect()로 연결 요청
4. 커널이 3-Way Handshake 수행
5. 연결 완료 후 connect() 반환
6. write()/send 계열로 데이터 전송
7. read()/recv 계열로 데이터 수신
8. close() 호출
9. 커널이 FIN/ACK 종료 과정 수행

중요한 점은 애플리케이션이 SYN과 ACK 패킷을 직접 만들어 보내는 것이 아니라는 점이다.
애플리케이션이 connect()를 호출하면 커널의 TCP 스택이 3-Way Handshake를 수행한다.

Mini-EDR에서 connect 시스템 콜과 SOCKADDR 레코드를 수집하는 것도 바로 이 연결 요청을 관찰하는 것이다.